NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

Experiment:- 5

Aim: To automate application deployment using GitHub/Jenkins.

LO mapped: LO3

LO: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

Theory:

Continuous Integration and Continuous Deployment (CI/CD) is a DevOps practice that automates the process of building, testing and deploying an application whenever changes are made to the source code. Instead of doing these steps manually every time, a tool keeps checking the code and runs the same sequence of actions automatically, which saves time and avoids human error.

Jenkins is an open-source automation server which is commonly used to set up these CI/CD pipelines. A pipeline in Jenkins is basically a set of stages that run one after another, and in most projects the three main stages used are Build, Test and Deploy. In the Build stage the application is compiled or prepared, in the Test stage automated tests are run to make sure the code is working correctly, and in the Deploy stage the final application is pushed to the server or environment where it will run.

Jenkins can be connected with GitHub so that the pipeline script does not have to be typed manually inside Jenkins every time. Instead, the pipeline is written as code in a file called Jenkinsfile, which is stored in the GitHub repository itself, and Jenkins simply pulls this file and executes it. To make the whole process fully automatic, GitHub can be configured to send a webhook (using the GitHub hook trigger for GITScm polling option) to Jenkins whenever a new commit is pushed. This way Jenkins does not need to be triggered manually — a build starts on its own as soon as new code is pushed to GitHub, and the updated application gets built, tested and deployed without any manual intervention.

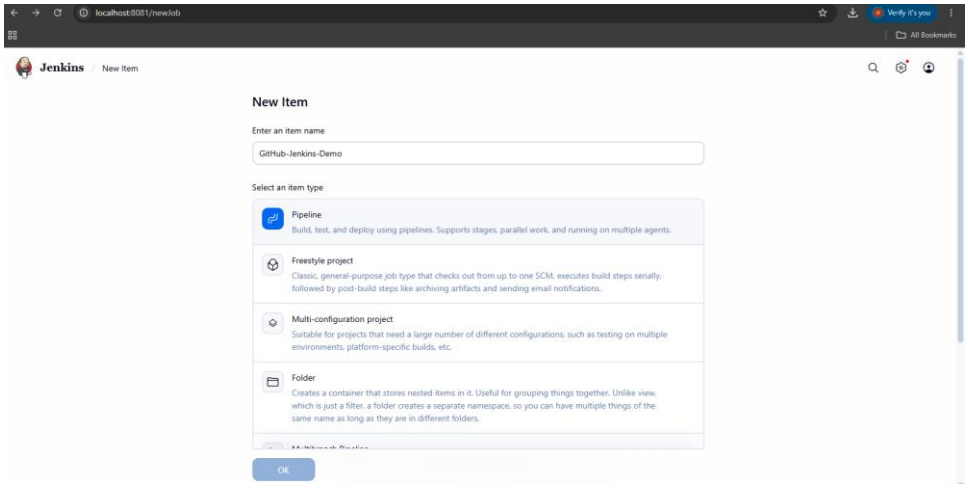
Steps:

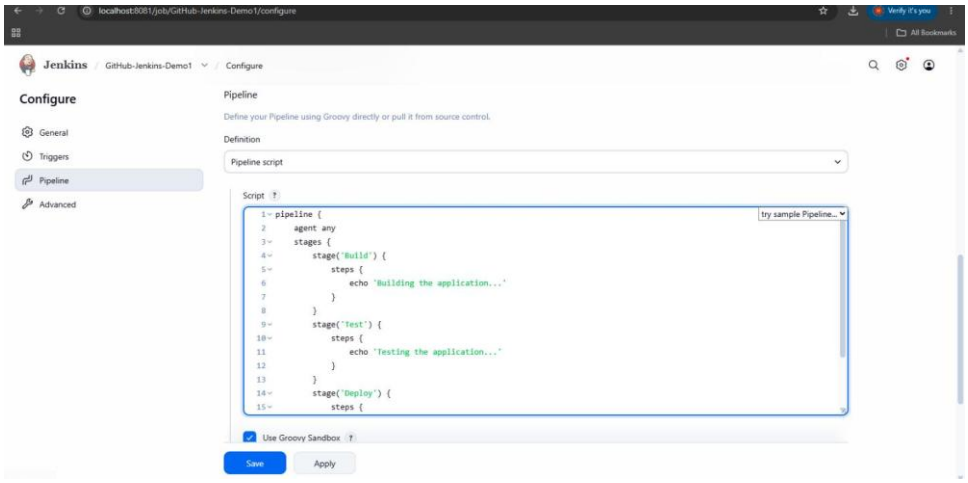
1. Open the Jenkins dashboard and create a new item of type Pipeline, and name it GitHub-Jenkins-Demo.
2. In the Configure page, under Pipeline > Definition, select Pipeline script and write a Groovy script with three stages — Build, Test and Deploy — each printing a message using echo. Save the job.
3. Click Build Now to run the pipeline manually for the first time and check the Console Output. The build finishes with status SUCCESS.
4. Create a new GitHub repository (github-jenkins-demo) and add an index.html file with basic content. Commit this change to the main branch.
5. Add a Jenkinsfile to the same GitHub repository, containing the same Build, Test and Deploy stages written as pipeline code, and commit it.
6. In Jenkins, go to Configure > Triggers and enable the GitHub hook trigger for GITScm polling option so that Jenkins can be triggered automatically whenever code changes are pushed to GitHub.
7. Change the Pipeline Definition to Pipeline script from SCM. Set SCM to Git, enter the GitHub repository URL, set the branch specifier to */main and the Script Path to Jenkinsfile, then save.

NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

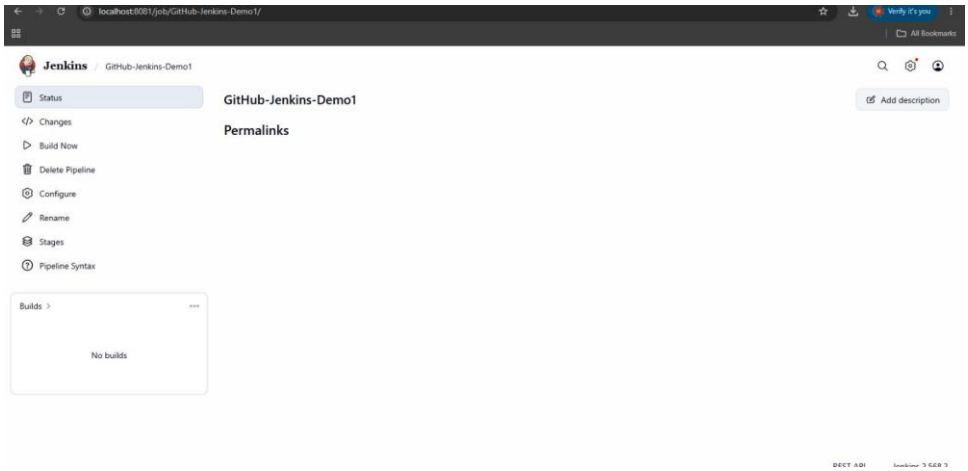
8. Jenkins now fetches the Jenkinsfile directly from GitHub instead of using a manually written script. Running the build clones the repository and executes the same Build, Test and Deploy stages, which can be verified from the console output.
9. To test automatic deployment, update index.html and the Jenkinsfile in GitHub (for example, change the content to Version 2.0) and commit the changes.
10. As soon as the commit is pushed, the GitHub webhook automatically triggers a new Jenkins build. Jenkins pulls the latest code and deploys the updated file, which is visible in the console output showing the new index.html content.
11. Open the deployed index.html file in the browser to confirm that the updated content (Version 2.0: Frontend Update Deployed Successfully!) has been deployed correctly, verifying that the CI/CD pipeline works end-to-end.

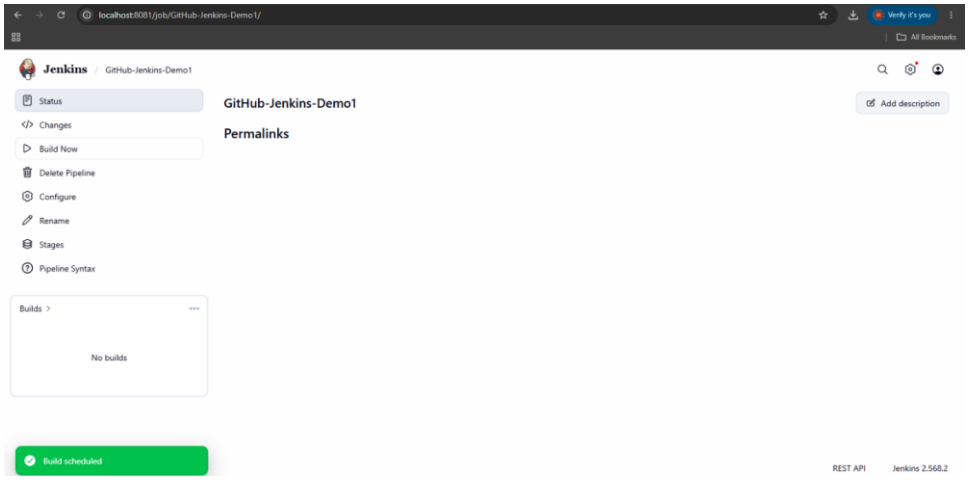
Output:

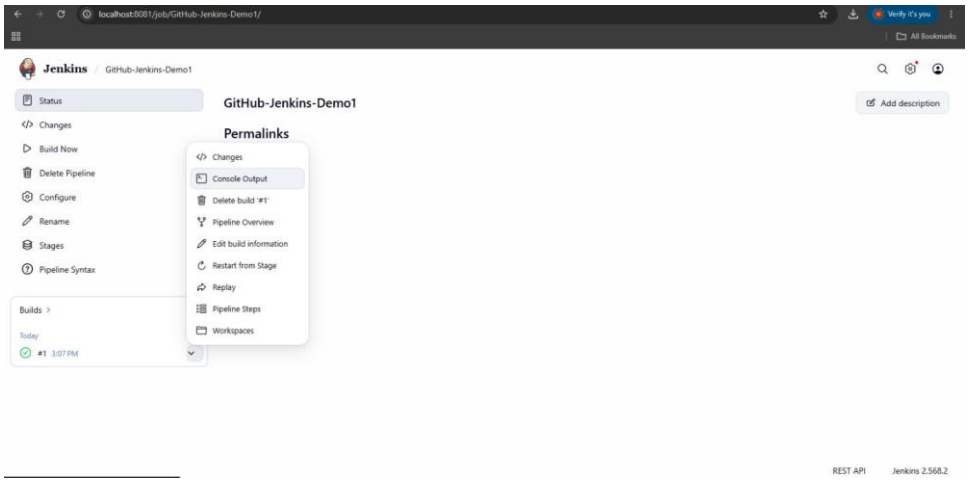




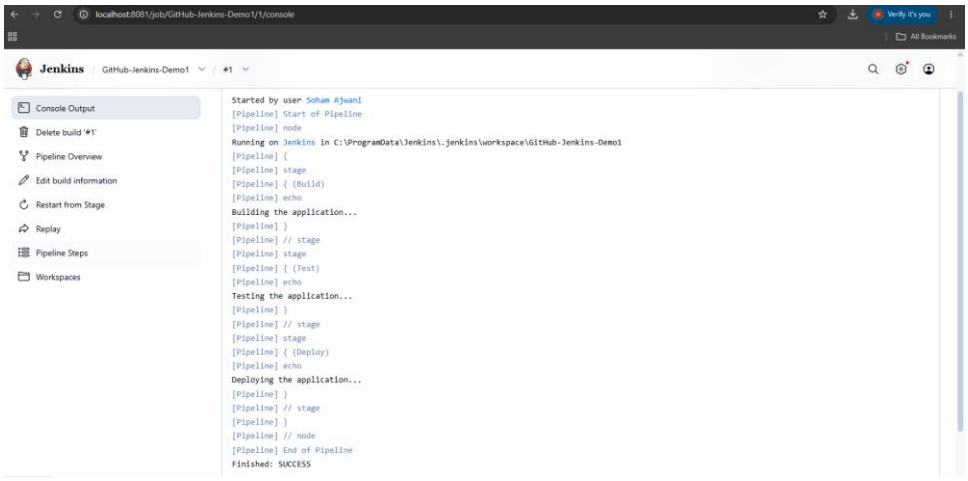
NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

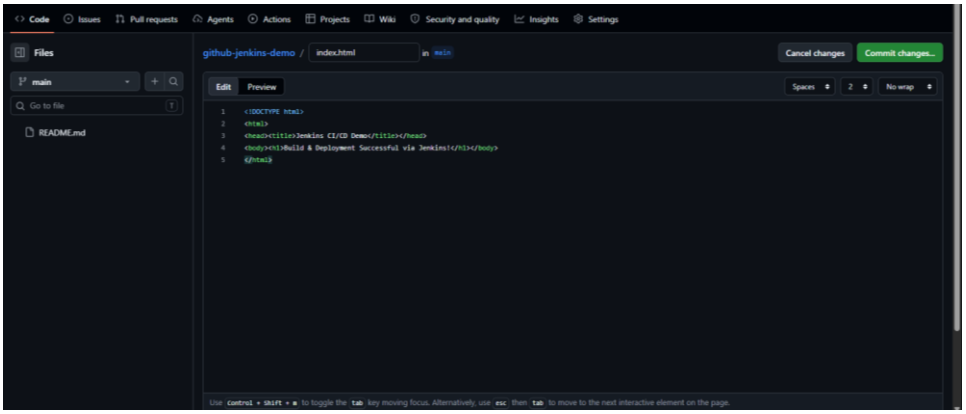


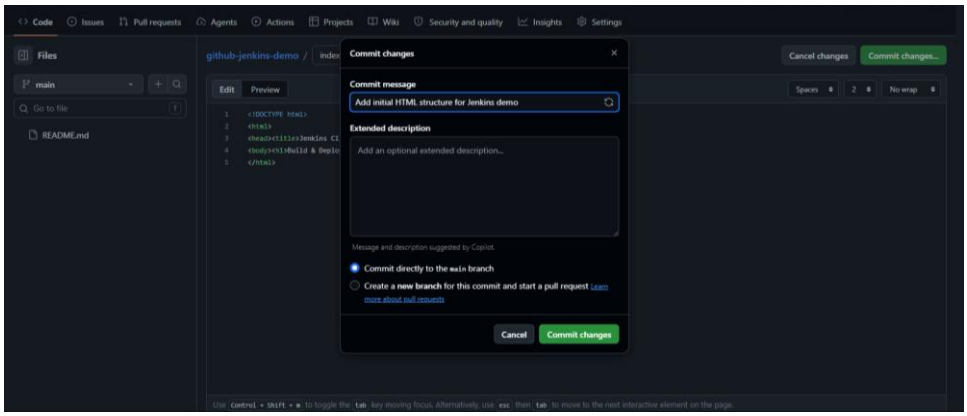


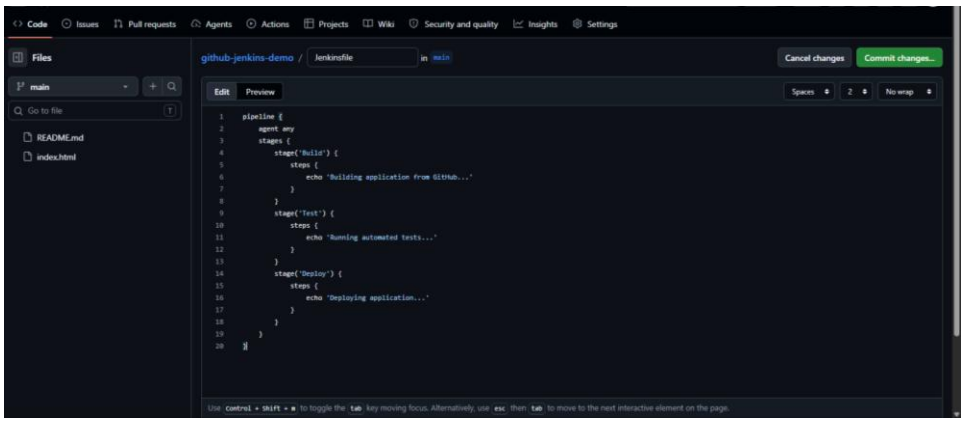


NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

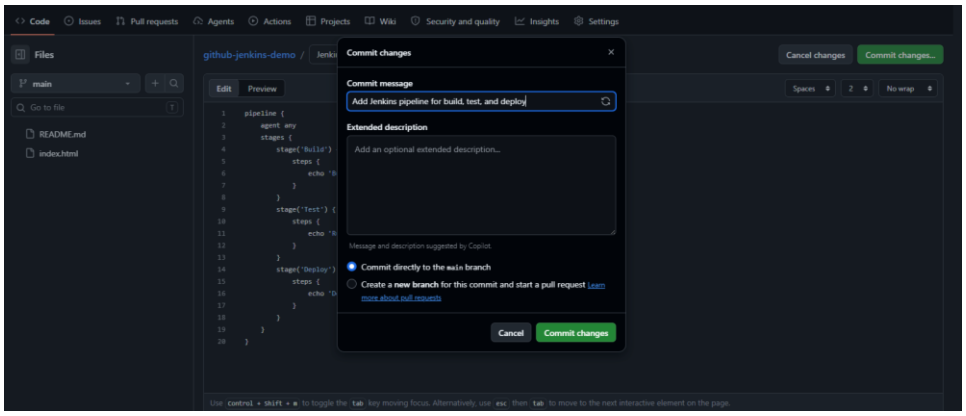




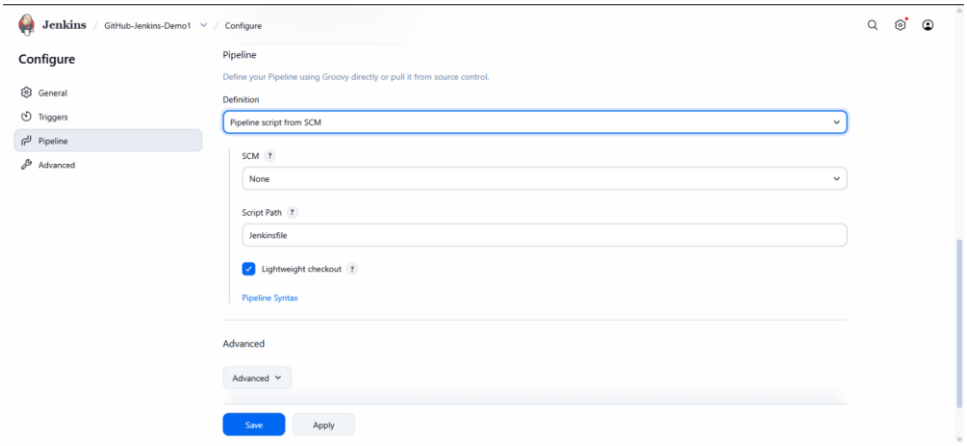


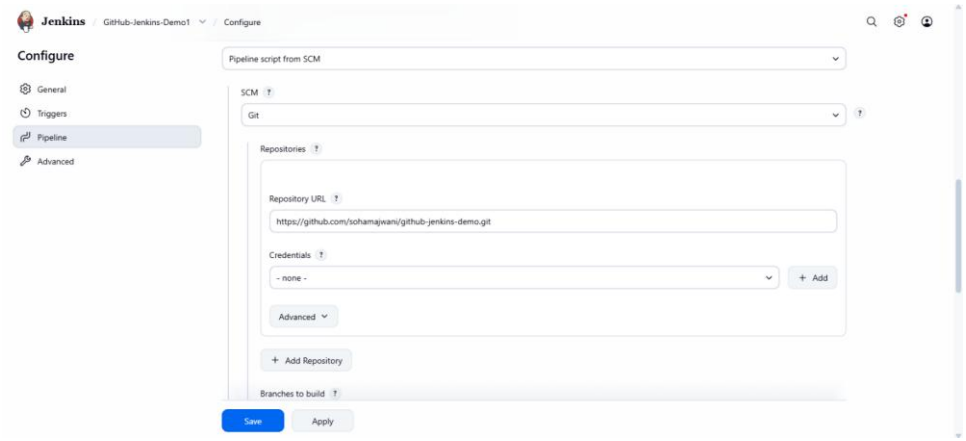


NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

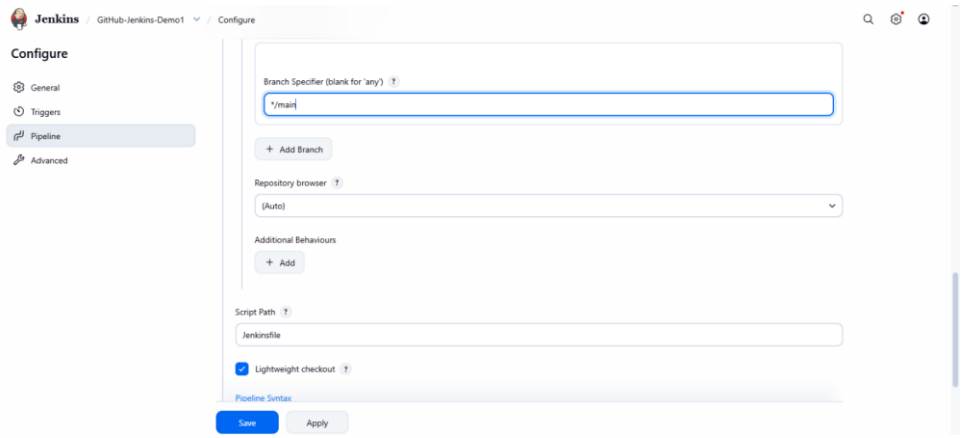


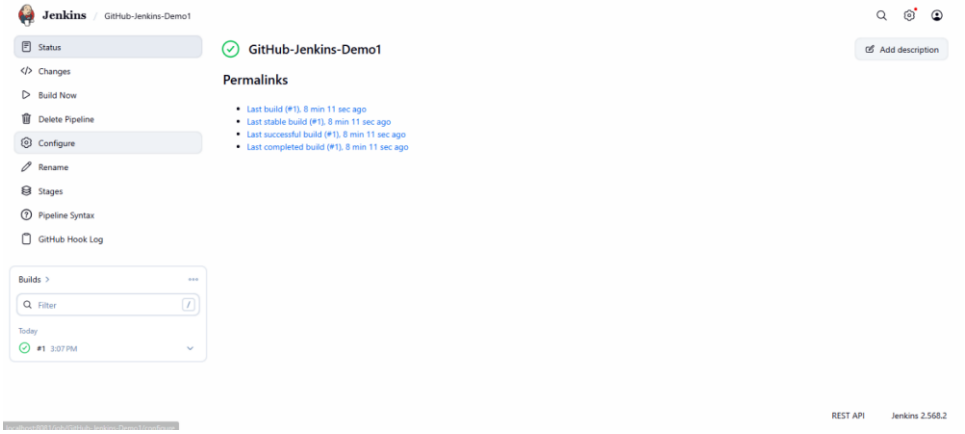


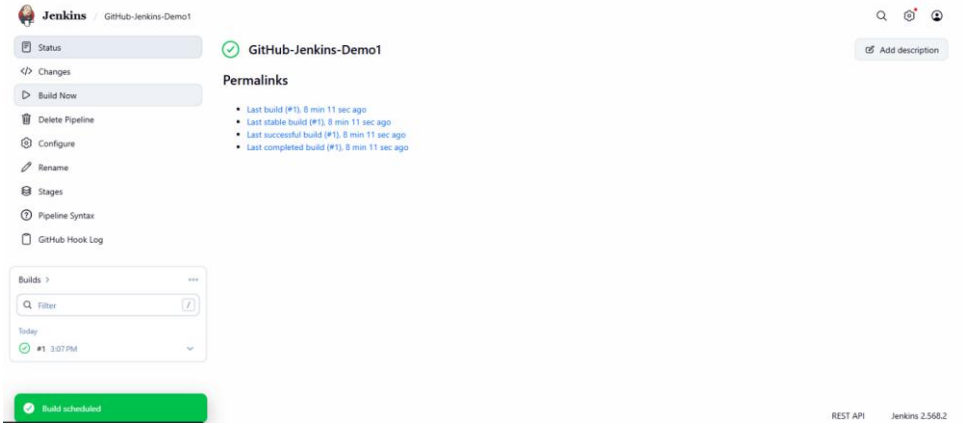




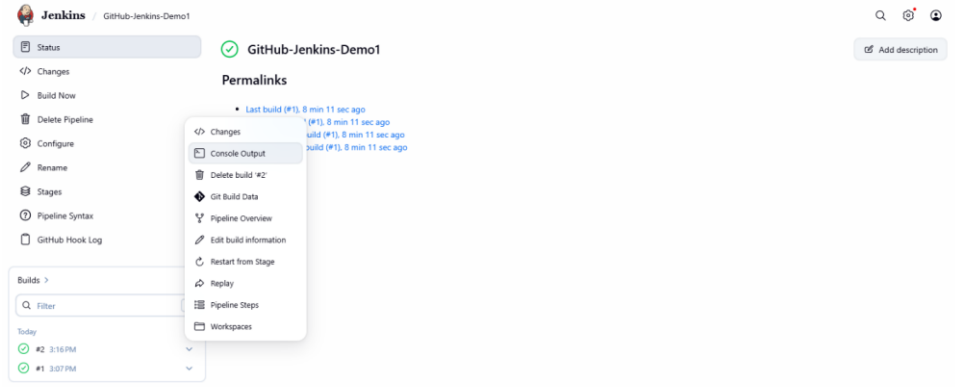
NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

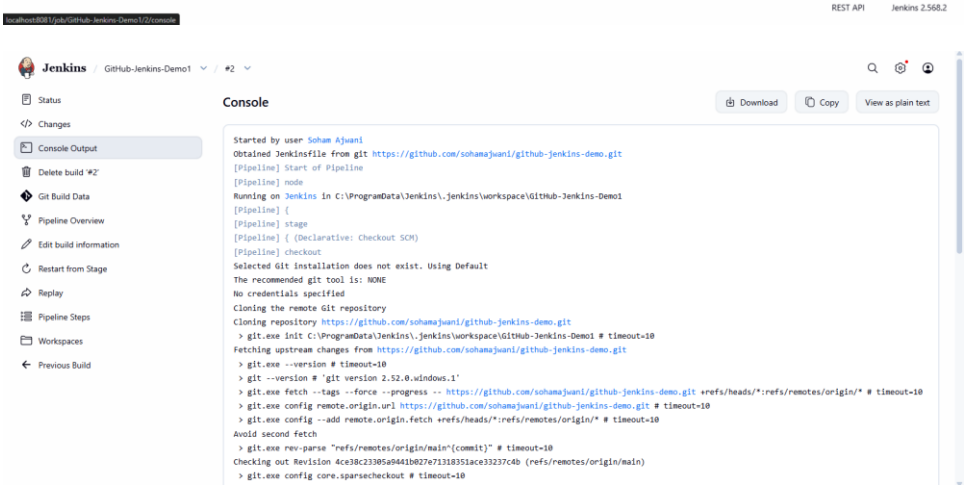


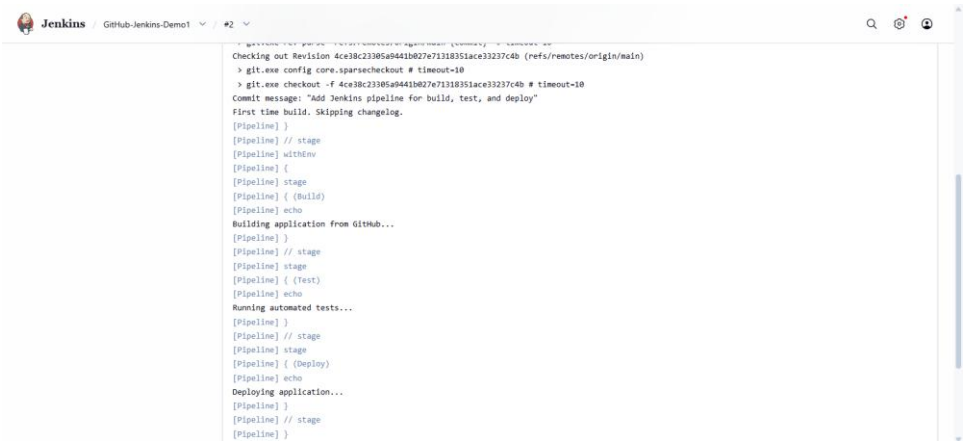




NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

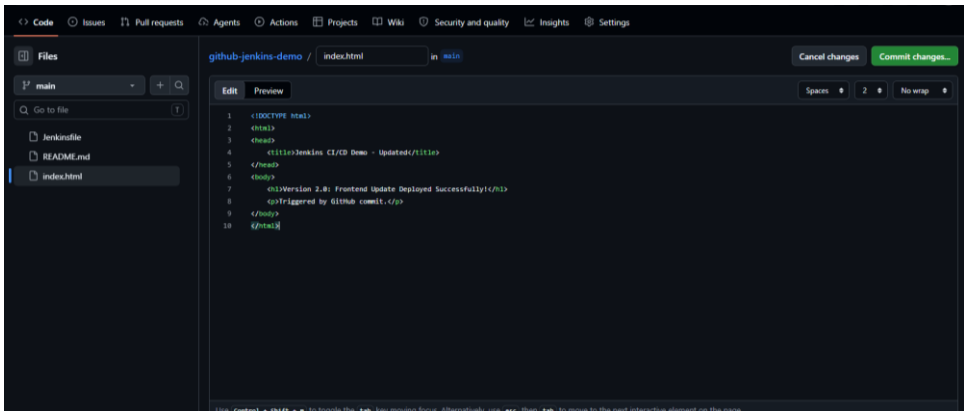


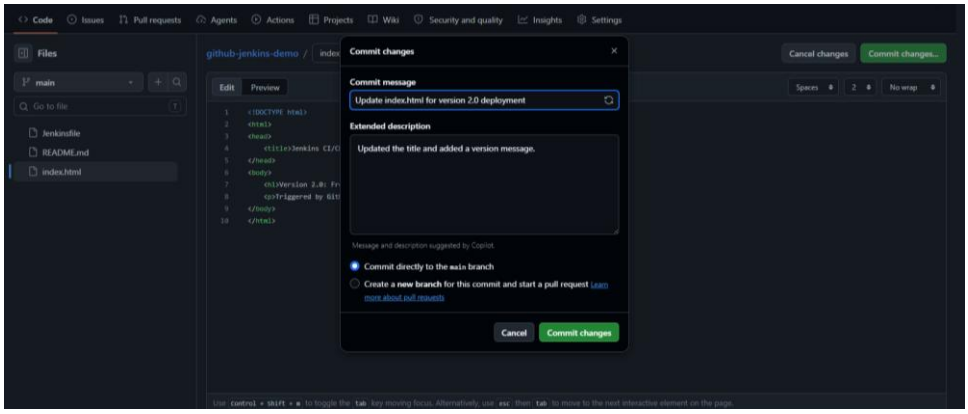


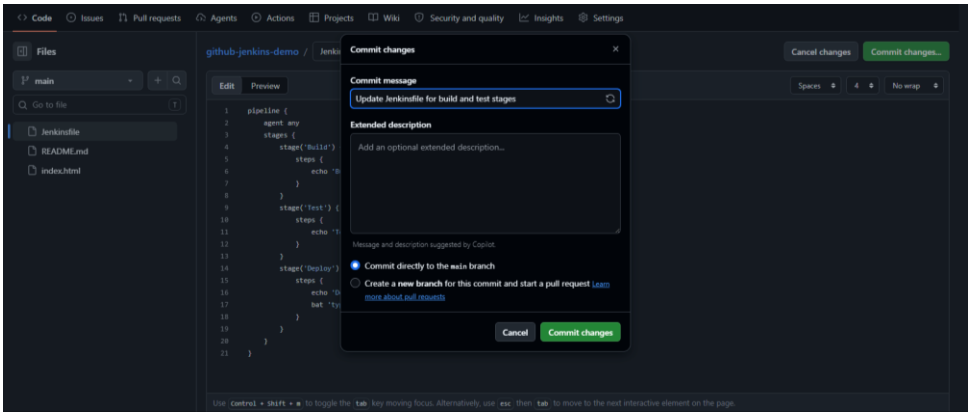


NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D

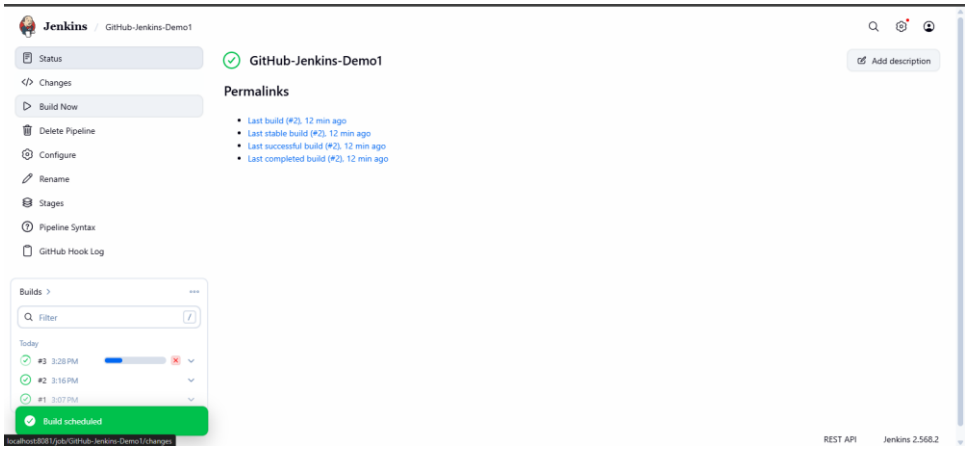


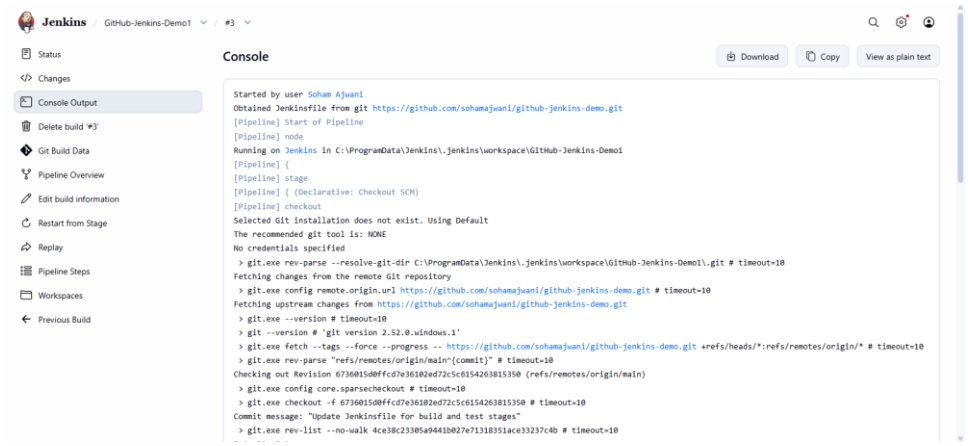






NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D







NAME:- HITEN PAHILAJANI
ROLL NO:- 2401069
DIV:- T1
BATCH:- T13
SUBJECT:- ASD&D



Conclusion:

In this experiment, an automated CI/CD pipeline was set up using Jenkins and GitHub. A Pipeline job was created in Jenkins with Build, Test and Deploy stages, first written manually inside Jenkins and later fetched directly from a GitHub repository using a Jenkinsfile. By enabling the GitHub hook trigger, Jenkins was configured to start a new build automatically whenever a commit was pushed to the repository. Updating the application code and pushing it to GitHub was enough to trigger a new build without any manual step, and the updated file was deployed successfully, which was verified by opening the deployed index.html page in the browser. This experiment shows how GitHub and Jenkins together help automate application deployment and reduce manual effort in the software development lifecycle.